HuGraph: An Economical Cloud-Native RDF Store

Bo Zhai
Hunan University
Changsha, China
zhaibo@hnu.edu.cn

Peng Peng
Hunan University
Changsha, China
hnu16pp@hnu.edu.cn

M. Tamer Özsu
University of Waterloo
Waterloo, Canada
tamer.ozsu@uwaterloo.ca

Lei Zou
Peking University
Beijing, China
zoulei@pku.edu.cn

Qian Lei
Hunan University
Changsha, China
lq127@hnu.edu.cn

ABSTRACT

The growing scale of RDF data has driven interest in cloud-native solutions that offer elastic scalability and cost efficiency. However, existing RDF stores are predominantly designed for integrated compute-storage architectures and fail to exploit the economic and operational benefits of modern cloud platforms—particularly storage disaggregation. In this paper, we present *HuGraph*, a cloud-native RDF store that minimizes monetary cost while delivering acceptable query performance under time constraints. HuGraph adopts a disaggregated architecture: at the storage layer, it organizes the RDF graph by grouping all triples that share the same property into a dedicated subgraph. Each such subgraph is stored as an independent object in low-cost object storage (e.g., Amazon S3), augmented with a novel *Range-Aware Offset Index* (RAOX) to enable efficient range-based data retrieval. At the computation layer, it models both the cost and the execution time of cloud storage operations. It formulates SPARQL query execution as a cost-minimization problem under time constraints—an NP-hard optimization task for which we propose an effective branch-and-bound algorithm. Experimental results show that HuGraph significantly reduces operational costs compared to traditional RDF stores, without sacrificing query performance in practical workloads.

1 INTRODUCTION

The *R*esource *D*escription *F*ramework (RDF) is a standard model for data exchange on the web. The fundamental unit of RDF is the *triple* (subject, property, object), which models a statement about a resource: the subject denotes the resource, the property denotes the relationship or attribute, and the object denotes the value or related resource. An RDF dataset is a collection of RDF triples and can be viewed as a graph. In this graph, subjects and objects serve as vertices, while triples form the edges, with properties acting as the labels on these edges. SPARQL (SPARQL Protocol and RDF Query Language) is the standard language used for querying and manipulating RDF graphs. A SPARQL query consists of *triple patterns*, similar to RDF triples, but with variables in place of subjects, properties, or objects. A SPARQL query can also be viewed as a query graph, and answering a SPARQL query involves finding subgraphs within the RDF graph that are homomorphic to the query graph [36].

As in their relational counterparts, managing RDF data on the cloud has become an attractive solution as data size grows. Cloud environments provide more cost-effective and elastic computing compared to traditional on-premise deployments. Cloud platforms allow for dynamic scaling of resources, and the large cloud vendors have major economies of scale relative to on-premise setups.

However, building a cloud-native RDF store presents unique challenges. Modern cloud architectures typically decouple storage and compute, allowing each layer to be scaled and billed independently. This architectural shift necessitates a fundamental rethinking of both RDF storage layout and query processing strategies. Moreover, while cloud computing is often promoted for its economic efficiency and operational flexibility, this very advantage elevates cost[1] to be a first-class design criterion along with performance. In many real-world applications, extreme query performance is secondary to predictable and minimized operational expenditure; thus, economic efficiency is a critical objective in system design. This gives rise to two core challenges:

(1) **Storage Design Challenge.** Designing an efficient and cost-effective RDF graph storage and indexing scheme that fully leverages cloud-native storage services.
(2) **Query Processing Challenge.** Developing a SPARQL query processing engine that operates in a storage-disaggregated architecture and aligns with the I/O patterns and cost structure of cloud storage, with the goal of minimizing cost while delivering acceptable query performance.

Existing systems fall short in addressing these requirements. Traditional RDF stores—such as Jena [16], RDF4J [2], Virtuoso [13], and gStore [35]—are built on an integrated compute-storage architecture. When deployed directly in the cloud, they require fixed resource allocation, making it difficult to scale dynamically in response to workload fluctuations. This often results in underutilized resources and unnecessary expenditure. Conversely, modern cloud-native relational DBMSs—such as Presto [27], Snowflake [12], Redshift Spectrum [5], and Apache Iceberg [1]—typically adopt storage disaggregation that separates computation and storage, allowing each to be independently managed and connected via a network. However, there are not many cloud-native DBMSs that are optimized for RDF data or SPARQL query semantics, leaving a gap in the landscape of RDF data management on the cloud.

In this paper, we propose a cloud-native RDF store, *HuGraph*, designed to minimize costs while meeting the performance requirements of user queries. Similar to most cloud-native DBMSs, we adopt the disaggregated architecture that divides computation and storage into separate layers connected through the network.

[1] In this paper, unless otherwise noted, "cost" refers to monetary cost.

At the storage layer, to address the storage design challenge, we use the remote shared object storage (e.g., Amazon's Simple Storage Service (S3) or Alibaba's Object Storage Service (OSS)) to store the RDF graphs, since cheap remote shared object storage provides order of magnitude cost savings over remote block stores like EBS, without significant performance disadvantages in mixed workloads [28]. We vertically divide the RDF graph into subgraphs where all triples with the same property are put into the same subgraph, and then each subgraph is organized as an independent file in the remote shared object storage. To enhance data access efficiency, we introduce an index structure called the *Range-Aware Offset Index* (RAOX). This index records the subject and object values of triples associated with a given property, along with their corresponding byte offsets within the respective file in the remote shared object storage. By leveraging this index, HuGraph supports six access methods that leverage the `Range GET` and `Range SELECT` operators natively offered by cloud object storage. We analyze the cost and latency characteristics of these methods and propose a cost model that captures both execution time and economic cost.

At the computation layer, when a SPARQL query is submitted with a specified time constraint, the problem of query processing can be formulated as an optimization problem aimed at minimizing cost while satisfying the time requirement. We prove that this problem is NP-hard and propose a branch-and-bound algorithm to determine the optimal query execution strategy.

Contributions. We make the following contributions in the paper.

- We propose a new RDF store, *HuGraph*, designed specifically for cloud computing environments, utilizing a disaggregated storage-computation architecture.
- At the storage layer, we leverage remote shared object storage to reduce costs and introduce the *Range-Aware Offset Index (ROAX)* to accelerate data access. We analyze the access methods and develop a model that captures both execution time and economic cost.
- At the computation layer, we formalize query processing and design a branch-and-bound algorithm that yields optimal execution strategies.
- We evaluate *HuGraph* on synthetic and real-world RDF graphs across multiple cloud platforms. Experimental results show that our system significantly lowers cost compared to existing RDF stores, without meaningful performance degradation on the tested workloads.

2 BACKGROUND

2.1 Preliminaries

In RDF, resources on the web are uniquely identified by Internationalized Resource Identifiers (IRIs). Using this concept, we define the RDF graph as follows, following the definitions from [21].

Definition 2.1. ***(RDF Terms, Triple and Graph)**. The set of RDF terms consists of the set of IRIs $\mathcal{U}$, the set of blank nodes $\mathcal{B}$ and the set of literals $\mathcal{L}$. A triple $(s, p, o) \in (\mathcal{U} \cup \mathcal{B}) \times \mathcal{U} \times (\mathcal{U} \cup \mathcal{B} \cup \mathcal{L})$ is called an RDF triple, where s is the subject, p is the property and o is the object. A set $\mathcal{T}$ of triples is called an RDF graph.*

Subject	Property	Object
McTaggart	deathPlace	London
Wittgenstein	deathPlace	England
McTaggart	mainInterest	Ethics
Grice	mainInterest	Epistemology
Ayer	mainInterest	Epistemology
Moore	mainInterest	Ethics
McTaggart	deathDate	1925-01-18
Wittgenstein	deathDate	1951-04-29
Ayer	birthPlace	London
Moore	birthPlace	London
Grice	birthPlace	England
Moore	influencedBy	Ayer
Moore	influencedBy	McTaggart
Grice	influencedBy	Ayer
Ayer	influencedBy	Wittgenstein

Figure 1: Example RDF graph

An RDF dataset can be represented as a graph where subjects and objects are vertices and triples are labeled edges.

Meanwhile, SPARQL is a structured query language over RDF where the primary building block is the basic graph pattern (BGP). In this study, we focus on BGP queries as they are foundational to SPARQL, and we consider techniques for handling them.

Definition 2.2. ***(Basic Graph Pattern)** Let $\mathcal{V}$ be a set of variables; variables bind to RDF terms from $\mathcal{U} \cup \mathcal{B} \cup \mathcal{L}$. A triple in $(\mathcal{U} \cup \mathcal{V}) \times (\mathcal{U} \cup \mathcal{V}) \times (\mathcal{U} \cup \mathcal{V} \cup \mathcal{L})$ is called triple pattern. We omit blank nodes from triple patterns for ease of exposition. $\mathcal{P}$ is the set of all triple patterns. Then, a basic graph pattern (BGP) is a set $Q \subset \mathcal{P}$. A set of queries is $\boldsymbol{Q} \subset 2^{\mathcal{P}}$.*

Example 2.1. *Figures 1 and 2 illustrate an example RDF graph and a SPARQL BGP, respectively. These examples are extracted from the well-known DBpedia RDF graph [17] and represent factual information about several English philosophers.*

For a BGP, the set of triples from an RDF graph that satisfy all its triple patterns is referred to as its matches. The formal definition is provided as follows.

Definition 2.3. ***(BGP Match)** Here, we denote the partial function $\mu : \mathcal{V} \rightarrow \mathcal{U} \cup \mathcal{B} \cup \mathcal{L}$ as a* mapping *μ from $\mathcal{V}$ to $\mathcal{U} \cup \mathcal{B} \cup \mathcal{L}$. Abusing notation, for a triple pattern e, we denote by $\mu(e)$ the triple obtained by replacing the variables in e according to μ.*

The set of matches for Q is denoted as $[\![Q]\!]$.

A SPARQL can also be seen as a query graph. We assume that query Q is a weakly connected directed graph; otherwise, each connected component of Q is considered separately.

2.2 Cloud Environment

2.2.1 Pricing of Cloud Services. Since the primary design goal of *HuGraph* as a cloud-native RDF store is to minimize costs while

```
SELECT ?x ?y ?z WHERE{
 ?x influencedBy ?y.
 ?y deathDate 1951-04-29.
 ?z mainInterest 'Ethics'.
 ?x birthPlace 'London'.
 ?y influencedBy ?z}
```

Figure 2: Example basic graph pattern query

Table 1: AWS service prices

Service	Specification	Price (US$)
S3 Storage	On demand	0.023/Month/GB
EBS Storage	On demand	0.08/month/GB
S3 PUT	On demand	0.005/1000 requests
S3 GET	On demand	0.0004/1000 requests
S3 SELECT	Scan	0.002/GB
	Return	0.0007/GB
EC2	2C2G	0.0208/hour
	2C4G	0.0464/hour
	2C8G	0.096/hour
	4C16G	0.192/hour
	8C32G	0.384/hour
	32C128G	1.536/hour
	64C256G	3.072/hour
Neptune Computing	2C4G (Standard)	0.098/hour
Neptune Storage	On demand	0.1/month/GB
Neptune Storage IO	On demand	0.2/million IOs
VPC	On demand	0.01/endpoint/hours

meeting performance requirements, it is essential first to consider the pricing of the cloud services used in its implementation. Thus, we begin by listing the relevant service prices, using Amazon Web Services (AWS) as a representative, since our implementation uses AWS services. However, HuGraph is designed to be deployable across different cloud providers that offer scalable object storage and virtualized compute services comparable to AWS. To demonstrate this flexibility, we validate HuGraph on Alibaba Cloud in our experiments. Table 1 provides an overview of AWS service pricing.

Amazon S3. Amazon's Simple Storage Service (S3) is a highly scalable and durable object storage service for storing and retrieving any amount of data from anywhere on the web. Within S3, data is stored as objects, typically in formats like CSV or Parquet. One of the key advantages of Amazon S3 is its low cost. As shown in Table 1, the storage cost is only $0.023 per GB per month, making it significantly lower than other storage services.

Data in S3 can be accessed using several methods, including the `GET` operator and the `SELECT` operator. Further, both `GET` and `SELECT` operators can invoke the `Range` functionality to form `Range GET` or `Range SELECT` operators, which allows users to download or scan specific byte ranges of an object, avoiding the need to retrieve and scan the entire file.

The `PUT` and `GET` operator as core methods of S3 for storing and retrieving objects in the cloud with obvious semantics (`GET` retrieves an entire file). These operations are very cheap. Regardless of the amount of data involved, the `GET` operator costs just $0.0004 per 1,000 requests, making the associated costs negligible in most practical applications.

The `SELECT` is a paid service that enhances the capabilities of querying data in S3. By scanning the data within files and returning only the data that matches the specified query criteria to the computing node, it introduces limited support for SQL query syntax. Currently, `SELECT` supports select, projection, and aggregation queries. Its performance is notably better than standard `GET` operator. However, `SELECT` operator incurs additional costs, which are based on the amount of data scanned and returned. As shown in Table 1, the price for scanning data is $0.002 per GB, and for returned data, the price is $0.0007 per GB.

Amazon EC2. Amazon EC2 (Elastic Compute Cloud) is a core service in AWS that provides scalable virtual servers, called instances, to run applications in the cloud. EC2 provides a wide range of instance types tailored to different workloads, each with varying pricing models. Table 1 lists the instance types and their corresponding costs used in our experiments.

Amazon EBS. Amazon EBS (Elastic Block Store) provides persistent block storage for use with Amazon EC2 instances. It offers low-latency, high-performance storage and is ideal for any application requiring raw storage access. As shown in Table 1, the storage cost is $0.023 per GB per month. To access data, the EBS volume is often attached to an EC2 instance and mounted to a directory. EBS volumes act as block storage, and after mounting, they can be used like local hard drives. Accessing data stored in EBS through an EC2 instance does not involve any additional costs.

Neptune. Neptune [8] is a fully managed graph database service offered by AWS that can support RDF graphs. Neptune should be deployed within a Virtual Private Cloud (VPC), which can provide network isolation. While Neptune simplifies the process of building and managing graph database services, it comes with a high price. As shown in Table 1, the total cost of using Neptune consists of four components: computing, storage, storage I/O, and the cost of an EC2 instance to access the Neptune service. These combined factors result in a relatively high overall cost.

2.2.2 Disaggregated Storage and Computation Architecture. Cloud data centers have moved to disaggregated storage architectures that better align with their goal of elasticity. This architecture allows for the independent scaling and upgrading of computation and storage resources. It optimizes resource utilization and meets the demand for simple, efficient shared storage in modern distributed applications, ultimately reducing the system's costs.

In storage disaggregated cloud-native systems, the storage layer often leverages cloud-native storage services like Amazon S3 or Amazon EBS, while the computation layer can be built using services such as Amazon EC2. For the storage layer, Amazon S3 offers virtually unlimited capacity, and EBS volumes can be dynamically resized to accommodate growing data needs. On the computation layer, Amazon EC2 instances can be dynamically launched or terminated based on workload requirements, with multiple pricing models (On-Demand, Reserved Instances, Spot Instances) available to optimize costs according to workload characteristics.

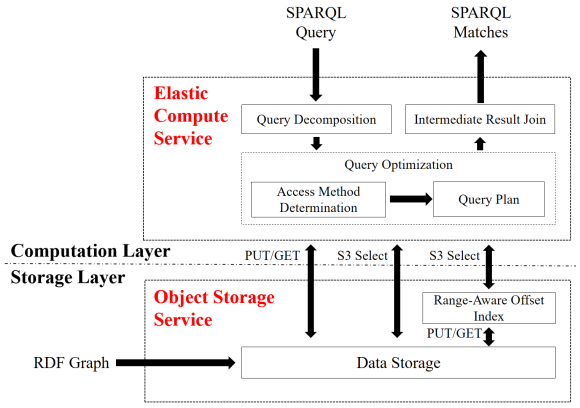


Figure 3: System Overview

3 SYSTEM OVERVIEW

Figure 3 presents an overview of *HuGraph*, which also follows the disaggregated storage architecture and consists of two layers: the storage layer and the computation layer. The storage layer is responsible for storing RDF graphs in the cloud, while the computation layer processes user-submitted SPARQL queries. The storage layer utilizes the cost-effective cloud object storage services such as Amazon S3 due to its low cost although it has slightly lower performance in mixed workloads [28]. The computation layer is built on the elastic cloud compute services like Amazon EC2, a widely adopted service in many cloud-native applications. Importantly, as noted earlier, *HuGraph* is cloud-agnostic: while our implementation uses AWS services, it can be readily deployed on any cloud platform that offers scalable object storage and virtualized compute services comparable to AWS. To demonstrate this flexibility, we also validate *HuGraph* on Alibaba Cloud in our experiments.

At the storage layer, the key design challenge is how to organize RDF graphs into S3 objects. Specifically, all triples sharing the same property are grouped into a subgraph. Each subgraph is then organized as an object in S3, with the property serving as the key, enabling users to efficiently retrieve all triples associated with that property. Additionally, RAOX is designed to further optimize access performance. Leveraging the characteristics of S3 while using the ROAX, we can find that there are six access methods with different characteristics. The analysis of these methods leads us to design a model that captures both execution time and economic cost. The details of the storage layer are discussed in Section 4.

At the computation layer, the input SPARQL query is executed on elastic cloud compute services such as Amazon EC2. A query is first decomposed into several subqueries, each of which can be processed directly on the subgraphs stored in S3. Because multiple access methods are available for accessing RDF data in S3 and they have different costs and performance characteristics, we select the optimal query execution strategy that balances the cost and performance, which is described in Section 5. Additionally, we demonstrate that finding the optimal query execution plan is NP-hard and propose a heuristic algorithm to achieve efficient query planning.

4 STORAGE LAYER

In this section, we describe our approach to storing RDF graphs on cloud platforms. First, we present our basic storage design for organizing RDF graphs on Amazon S3. Then, we introduce the index designed to speed up data access within S3.

4.1 Basic Storage Design

S3 is an object-oriented system, and the main design challenge is how to organize the RDF graphs as objects within S3. Storing the entire RDF graph as a single object is not a good choice, as query processing often requires multiple accesses to various parts of the graph. Storing the graph as one object indicates that each access necessitates scanning the entire graph, which is inefficient.

A more effective approach is to divide the large RDF graph into smaller subgraphs, with each subgraph organized as an independent object. Because the number of properties is generally limited and SPARQL query variables rarely appear in property positions [3], we can define each subgraph as the triples sharing the same property. We group triples by property, resulting in one subgraph per property. Each such subgraph is then organized as a single object in Amazon S3, where the property name serves as the object key, and each subject–object pair is encoded as a row within the object.

To further enhance retrieval locality, we sort these subject–object pairs by their object values. This optimization leverages the observation that object positions in real-world SPARQL queries are frequently constants.

Example 4.1. *The example RDF graph in Figure 1 consists of five properties:* mainInterest, birthPlace, influencedBy, deathPlace, *and* deathDate. *Each property maps to a separate subgraph, which is then organized as an individual S3 object, as shown in Figure 4.*

To improve performance, in our implementation, we use a standard approach (see, e.g., [20]) of replacing all IRIs and literals with unique integer IDs via a mapping dictionary, which can compress the triple store by reducing triples to ID form and simplify query processing to enable faster and more streamlined operations. The trade-off for these benefits is the need for two additional dictionaries. During query parsing, IRIs and literals in the query are converted to their corresponding dictionary IDs using these mappings. After query execution, the resulting IDs are converted back to their original lexical forms before the results are returned.

The mapping dictionaries are also stored as an S3 object offline and loaded onto an EC2 instance when it is initialized for query processing. To maintain the dictionaries and enable fast ID lookups within the EC2 instance, we use a lightweight key-value store library like LevelDB[2]. The costs associated with converting IRIs and literals to their dictionary IDs and vice versa are generally low, while query processing operations like joins typically dominate the overall processing time.

4.2 Range-Aware Offset Index

As mentioned earlier, there are two primary methods available for accessing data in S3: the `GET` and the `SELECT` operator. Using only the above basic storage design to retrieve all matches for a

[2]https://github.com/google/leveldb

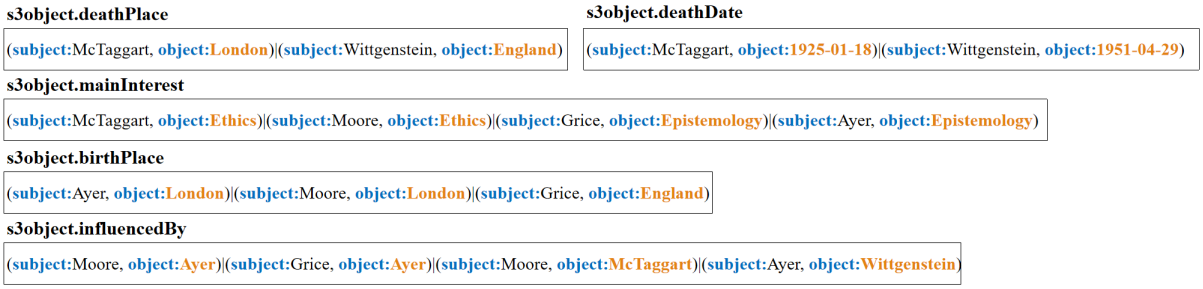


Figure 4: Organization of an RDF graph into S3 objects based on properties (Sorted by objects)

triple pattern with property p in a query presents trade-offs: the GET operator retrieves all triples with property p at a low cost but requires significant processing time for such a large volume of data. The SELECT operator can improve performance by pushing filtering down to the storage layer, but it incurs higher costs. To leverage the strengths of both methods, we design the ROAX as an index that combines the GET and SELECT operators, balancing cost and performance.

The ROAX centers on the Range GET and Range SELECT operators. These operators are defined as extensions of the standard GET and SELECT operators, augmented with a Range parameter that constrains the byte range of data to be scanned during access.

For the object in S3 corresponding to property p, we maintain the index file size and an index table that records the values of subjects and objects for triples with property p, as well as the byte offsets of these triples within the object. Thus, the metadata of a ROAX file consists of the *File Header* (*Size*) and the *Index Header* (*Value*, *Position*, *StartOffset*, *EndOffset*):

- *Size*: the number of bytes of the file object,
- *Value*: the value of either the subject or object in a triple,
- *Position*: an indicator specifying whether the value corresponds to the subject or the object,
- *StartOffset* and *EndOffset*: the byte offsets that specify the boundaries of the location where triples containing the given *value* in the specified *position* are stored within the file object.

Given a property p, its ROAX is denoted as $ROAX(p)$ with the file size represented as $\text{Size}(ROAX(p))$. Each record consisting of an index $r \in ROAX(p)$ is a tuple of the form $(r.Value, r.Position, r.StartOffset, r.EndOffset)$. Additionally, for each property p, its index also stores the size of $ROAX(p)$ as metadata for cost estimation during query processing and optimization.

Then, accessing the object file associated with property p using ROAX involves two phases. In the first phase, a property over the subject or object is evaluated by querying the index via a SELECT or a GET operator. This retrieves the byte offsets of relevant subjects or objects, which are then passed to the computation layer. In the second phase, these retrieved byte offsets are used to directly fetch the corresponding subject–object pairs from the object file associated with property p. This is accomplished using Range GET or Range SELECT operator, both of which allow for efficient retrieval by scanning only the necessary portions of the data, thereby minimizing data transfer and reducing overall query cost.

The ROAX is also stored as an S3 object, with each index file corresponding to a property and its corresponding S3 object. The index file for a property contains the total size of its corresponding S3 object, as well as the starting and ending row positions for specific subject or object values. By scanning the index or querying it via SELECT, we can obtain the byte offsets of the subject or object values and estimate the size of the data to be returned. This estimated size can be used to estimate query time and cost, while the `byte offset` serves as a parameter for the Range in the S3 request. If the query does not contain a subject or object constant, the total size of the S3 object recorded in the index is used to estimate the query time and cost.

Clearly, creating additional indices for more properties and subject or object values requires more storage space and increases the time needed to scan the indices, which in turn reduces query performance. Thus, only a subset of subject/object values should be selected for indexing. Intuitively, subject/object values with higher selectivity should be prioritized for indexing, because indexing them can greatly reduce the volume of data that need to be scanned during query execution. Thus, when constructing the index, we incrementally add data entries in order of decreasing selectivity.

Definition 4.1. ***(Selectivity)*** *Given a property p and the set $\mathcal{T}_p$ of triples with the property p, the selectivity for a subject or object value v is defined as the ratio of the total number of triples to the number of result values, where the property is p and the subject or object is v:*

$$s_p(v) = \frac{|\mathcal{T}_p|}{|\mathcal{T}_p(v)|} \tag{1}$$

where $|\mathcal{T}_p(v)|$ denotes the set of triples in $\mathcal{T}_p$, where the property is p and the subject or object is v.

To ensure the effectiveness of these indices, the combined size of the scanned index file and the queried data file should be smaller than the size of the entire data file. If this condition is not met, using the index may provide no advantage over directly scanning the object file in S3. To achieve this, the combined size of the index itself and the largest data segment it references in the original file must remain smaller than the total size of the original data.

Conversely, if the index is too small, it may include only a limited subset of values selected for indexing, resulting in a low hit rates during query processing. This reduces the effectiveness of the index, as most queries still require scanning the base data. Thus, the size of the index must be carefully balanced: it should be large enough to

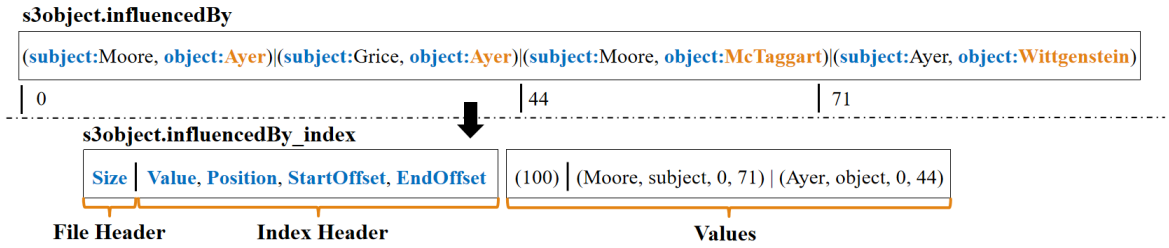


Figure 5: Example range-aware offset index for property $influencedBy$

provide good coverage (high hit rate), yet small enough to minimize access overhead.

Formally, for a given property p and the corresponding set $\mathcal{T}_p$ of triples associated with p, we define a condition for index usefulness. Let Size($ROAX(p)$) denote the size of the ROAX for p, and let the largest referenced data segment be determined by

$$\max_{r \in ROAX(p)} (r.\text{EndOffset} - r.\text{StartOffset}).$$

Then, the index is considered useful only if:

$$\text{Size}(ROAX(p)) + \max_{r \in ROAX(p)} \{r.\text{EndOffset} - r.\text{StartOffset}\} < \alpha \times \text{Size}(\mathcal{T}_p) \quad (2)$$

where Size($\mathcal{T}_p$) is the total size of all triples with property p, and $0 < \alpha \leq 1$ is a threshold parameter that controls the strictness of the condition. A smaller α enforces a stricter size constraint, favoring higher efficiency at the cost of potentially lower coverage. We discuss how to set α in Section 4.3.

Example 4.2. *Figure 5 illustrates the example ROAX for the property influencedBy which sorted by object element. The first index records that the total size of the corresponding S3 object is* 100 *bytes. The value Moore appears in the first and third triples of the object file, spanning from byte offset* 0 *to* 71*. Similarly, the value Ayer occurs in the first and second triples, with a byte range from* 0 *to* 44*. For Moore, an additional filtering step is required on the computation server to eliminate false positives and ensure correctness. In contrast, values such as McTaggart, Grice, and Wittgenstein appear only once. Creating index entries for them results in index files larger than the original object file, degrading query performance. Thus, no index entries are created for them.*

4.3 Access Methods

Leveraging the characteristics of S3 while using the ROAX, we can find that there are six access methods for accessing the relevant data stored in S3.

(1) **Direct GET (DG)**: Retrieve data directly from S3 using a standard GET operator.
(2) **Direct SELECT (DS)**: Query data directly using SELECT operator.
(3) **Indexed Range GET (GG)**:
   (a) Scan the index file using GET to obtain byte offsets of relevant subjects/objects.
   (b) Fetch the data from the object file using Range GET based on the retrieved offsets.
(4) **Indexed Range SELECT (GS)**:
   (a) Scan the index file using GET to obtain byte offsets.
   (b) Query the object file using Range SELECT with the obtained offsets.
(5) **SELECT Indexed Range GET (SG)**:
   (a) Use SELECT to scan the index file and retrieve byte offsets.
   (b) Access the object file via Range GET using these offsets.
(6) **SELECT Indexed Range SELECT (SS)**:
   (a) Use SELECT to scan the index file for byte offsets.
   (b) Query the object file via Range SELECT with these offsets.

Since only one access method need be employed per data retrieval task, it is crucial to evaluate their relative performance. Performance depends primarily on two factors: (1) the inherent latency and cost differences between GET and SELECT operators, and (2) the ROAX size corresponding to its property file.

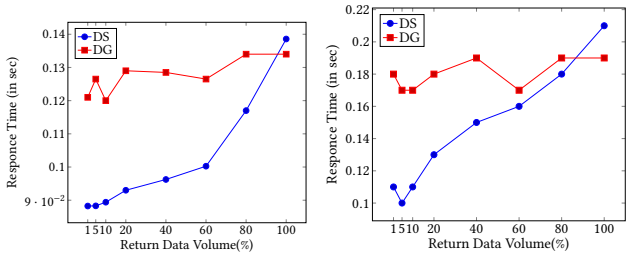
To evaluate the inherent latency and cost trade-off between GET and SELECT, we first conduct an experiment as follows. We select four properties from Wikidata whose corresponding S3 objects range in size from 1 MB to 1000 MB. For each property, we then access varying proportions of the data (from 1% to 100%) using SELECT and GET, respectively. Figure 6 and Figure 7 show how performance and cost vary with the result data size and the property file size for the two operators.

Figure 6 shows that the latency of a GET operator exhibits a linear positive correlation with the S3 object size, while remaining largely insensitive to the result data size. In contrast, a SELECT operator involves both data scanning and result retrieval, causing its latency and cost to scale with both the result data size and the S3 object size. Furthermore, for a fixed S3 object size, the latency of SELECT gradually approaches that of GET as the result data size increases, and eventually exceeds it when the result data size approaches the S3 object size. Based on this observation, for a triple pattern tp with property p and a value constraint v, we define its time cost $TC^{DG}(tp)$ and $TC^{DS}(tp)$ for DG and DS as follows.

$$TC^{DG}(tp) = |\mathcal{T}_p| \times T_{DL} \quad (3)$$

$$TC^{DS}(tp) = \frac{|\mathcal{T}_p|}{s_p(v)} \times T_{DL} + |\mathcal{T}_p| \times T_{DS} \quad (4)$$

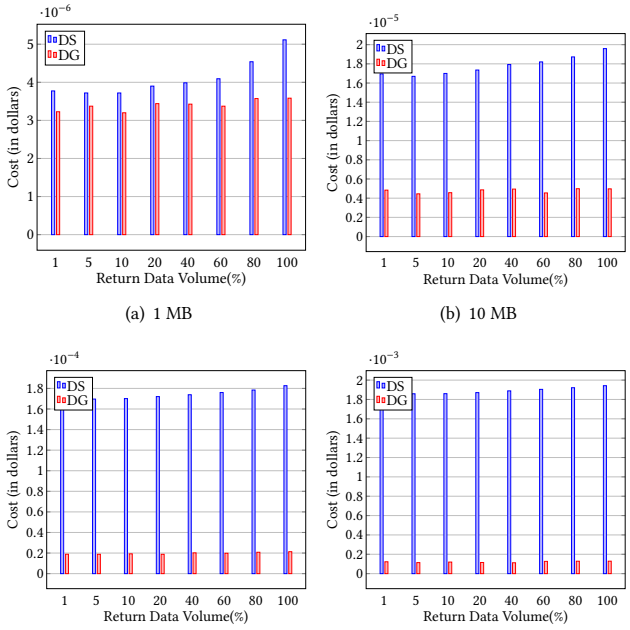
where T_{DL} denote the unit times of downloading and T_{DL} represent unit times of scanning a data unit via a SELECT operator, respectively. Since SELECT uses the same data transfer channel as GET for data download, the time required for the data transfer of SELECT is



(a) 1 MB (b) 10 MB

(c) 100 MB (d) 1000 MB

Figure 6: Performance comparison of GET and SELECT across varying S3 object sizes



(a) 1 MB (b) 10 MB

(c) 100 MB (d) 1000 MB

Figure 7: Cost Comparison of GET and SELECT Varying S3 Object Sizes

also T_{DL}. T_{DS} and T_{DL} can be obtained offline as the metadata and in practice $T_{DS} \ll T_{DL}$.

Using these definitions together with the $ROAX(p)$ of property p, we can now formulate the time costs of the ROAX-based access methods GG, GS, SG, and SS for a triple pattern tp with a property p and a value constraint v.

For GG, the index file is first downloaded via GET to obtain the byte offsets. The cost of retrieving the index is $Size(ROAX(p)) \times T_{DL}$. For a specific value v, the required data is fetched via Range GET using the start and end offsets recorded in $ROAX(p)$. Let $r(Value = v)$ be the corresponding record; the number of bytes to download is $(r(Value = v).EndOffset - r(Value = v).StartOffset)$, costing $(r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DL}$.

$$TC^{GG}(tp) = Size(ROAX(p)) \times T_{DL} + (r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DL} \quad (5)$$

For GS, the index file is also downloaded via GET at cost $Size(ROAX(p)) \times T_{DL}$. The object file is then queried using Range SELECT over the same byte range, which involves scanning the range and downloading the result tuples. The cost is therefore the scan cost $(r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DS}$ plus the result download cost $|\mathcal{T}_p(v)| \times T_{DL}$.

$$TC^{GS}(tp) = Size(ROAX(p)) \times T_{DL} + |\mathcal{T}_p(v)| \times T_{DL} + (r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DS} \quad (6)$$

For SG, the index file is scanned via SELECT to retrieve the byte offsets. The scan cost is $Size(ROAX(p)) \times T_{DS}$, and we denote by d_1 the small constant time required to download the offset parameters returned by the SELECT call. The object data is then obtained through a Range GET, costing $(r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DL}$.

$$TC^{SG}(tp) = Size(ROAX(p)) \times T_{DS} + d_1 + (r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DL} \quad (7)$$

Finally, for SS, the index file is scanned via SELECT at cost $Size(ROAX(p)) \times T_{DS} + d_1$. The object file is then accessed with Range SELECT, which scans the specified byte range and downloads the result tuples, leading to an additional cost of $(r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DS}$.

$$TC^{SS}(tp) = Size(ROAX(p)) \times T_{DS} + d_1 + |\mathcal{T}_p(v)| \times T_{DL} + (r(Value = v).EndOffset - r(Value = v).StartOffset) \times T_{DS} \quad (8)$$

The index file for a property already records the total size of its corresponding S3 object, so $Size(ROAX(p))$ is immediately available. As with T_{DS} and T_{DL}, the constant d_1 can be obtained offline. In this paper, we estimate $|\mathcal{T}_p(v)|$ and $(r(Value = v).EndOffset - r(Value = v).StartOffset)$ using the heuristics described by Stocker et al. [26]. Further optimizations for selectivity estimation are well-studied and orthogonal to our proposed methods; we therefore do not discuss them further in this paper.

When considering the economic cost results in Figure 7 alongside the latency findings, we observe that for large S3 objects, SELECT can offer performance advantages for small result sizes, but incurs a substantial scan cost that dominates the overall query cost. Meanwhile, as the S3 object size decreases, the computing cost becomes the dominant factor in the total query cost. Consequently, a trade-off emerges between the performance advantage of SELECT and the cost advantage of GET in such scenarios. Based on this observation, for a triple pattern tp with property p and a value constraint v, we define its economic cost $EC^{DG}(tp)$ and $EC^{DS}(tp)$ as follows.

$$EC^{DG}(tp) = TC^{DG}(tp) \times E_C + d_2, \tag{9}$$

$$EC^{DS}(tp) = TC^{DS}(tp) \times E_C + |\mathcal{T}_p| \times E_{DS} \tag{10}$$

where E_C denote the unit economic costs of Computing Layer during the time of data retrieving and E_{DS} represent unit economic costs of scanning a data unit via a SELECT operator, respectively. d_2 in definition represents the request cost for GET and it is always very small. These unit costs can be obtained from cloud service provider pricing list, and in practice $d_2 \ll E_C \ll E_{DS}$.

Following the $EC^{DG}(tp)$ and $EC^{DS}(tp)$ definitions, considering with the index $ROAX(p)$ of property p, we can define the economic costs $EC^{GG}(tp)$, $EC^{GS}(tp)$, $EC^{SG}(tp)$, and $EC^{SS}(tp)$ of a triple pattern tp with ROAX-based query access methods GG, SG, GS and SS.

For GG, two GET requests are issued—one for the index file and one for the object data—resulting in a request cost of $2 \times d_2$. Meanwhile, the computing resource cost during data retrieval is $TC^{GG}(tp) \times E_C$.

$$EC^{GG}(tp) = TC^{GG}(tp) \times E_C + 2 \times d_2 \tag{11}$$

For GS, one GET request first retrieves the index file at cost d_2. The object data is then accessed via Range SELECT, which scans a byte range of size $r(Value = v).EndOffset - r(Value = v).StartOffset$ and incurs a scan cost of $(r(Value = v).EndOffset - r(Value = v).StartOffset) \times E_{DS}$. The computing time cost is $TC^{GS}(tp) \times E_C$.

$$EC^{GS}(tp) = TC^{GS}(tp) \times E_C + d_2 + (r(Value = v).EndOffset - r(Value = v).StartOffset) \times E_{DS} \tag{12}$$

For SG, the index file is first scanned via SELECT at a cost of $Size(ROAX(p)) \times E_{DS}$. The object data is then fetched using a single GET request (cost d_2). The computing time cost is $TC^{SG}(tp) \times E_C$.

$$EC^{SG}(tp) = TC^{SG}(tp) \times E_C + Size(ROAX(p)) \times E_{DS} + d_2 \tag{13}$$

For SS, both the index file and the relevant byte range of the object file are accessed via SELECT. The total scanned data volume is $(Size(ROAX(p)) + (r(Value = v).EndOffset - r(Value = v).StartOffset))$, leading to a scan cost of $(Size(ROAX(p)) + (r(Value = v).EndOffset - r(Value = v).StartOffset)) \times E_{DS}$. The computing time cost is $TC^{SS}(tp) \times E_C$.

$$EC^{SS}(tp) = TC^{SS}(tp) \times E_C + (Size(ROAX(p)) + (r(Value = v).EndOffset - r(Value = v).StartOffset)) \times E_{DS} \tag{14}$$

In summary, the above experiment results suggest that for triple patterns with low selectivity (those that retrieve a substantial portion of an S3 object), methods relying on GET (e.g., DG) can achieve performance comparable to SELECT-based approaches while incurring significantly lower economic cost.

The discussion in Section 4.2 has already elaborated on how ROAX can effectively reduce the transmission and scanning volume of GET and SELECT access methods for data. Consequently, as defined in Equations (5) - (8) and Equations (11) - (14), the efficiency of ROAX depends on the effective index record size, which is controlled by the threshold parameter α.

We also illustrate this trade-off through an experiment. Figure 8 shows how performance and cost vary with the index record size and hit ratio for the six operators.

Figure 8 demonstrates that for a given property, both performance and cost are strongly correlated with the size ratio between the data object and its corresponding index file. Specifically, for any given property p, the performance of access methods that utilize the ROAX exhibits a trend of first improving and then degrading as α increases, while the cost increases monotonically. Therefore, for each property p, the optimal choice of the threshold parameter α is constantly remain on 40%-60%.

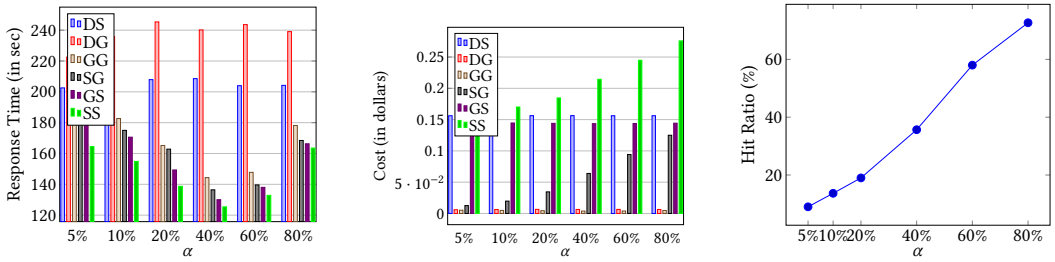
Further, based on the above cost models, since only one access method can be applied for each data retrieval, an *Access Method Determination* step is required to choose the optimal access method during query processing in the computation layer, which will be described in detail in Section 5.2.

4.4 Further Optional Optimizations

Beyond property-based data organization, additional optimization strategies also can be employed. For instance, one could create secondary S3 objects organized by subject or object values (SPO or OSP layouts). Such structures would accelerate triple patterns where constants appear in the subject or object positions. However, this approach introduces data replication, increasing storage and maintenance complexity. Therefore, the application of such techniques in HuGraph is directly related to the volume of data in secondary S3 objects and the number of such objects.

Specifically, generating additional SPO or OSP files for low-selectivity data (large volumes of data) introduces substantial data redundancy in storage, incurring non-negligible additional storage costs. As Figure 9(a) presented, OSP only contributes at most 10% of cost saving from 40% to 80% size ratio. Therefore, their practical effectiveness is significantly diminished due to the large size of these files when the subject or object in *ROAX*, even though triple patterns with constants in the subject or object position can be directly evaluated using the SPO or OSP. In contrast, during comprehensive queries, their contribution to cost becomes even smaller.

For highly selective data (small data volume), the resulting SPO or OSP files are much smaller than the property-partitioned data files. In such cases, as Figure 9(a) shows, SPO or OSP can provide effective query acceleration and cost saving for this subset of data. However, due to the large number of highly selective subjects or objects, and because for each such subject or object, every involved

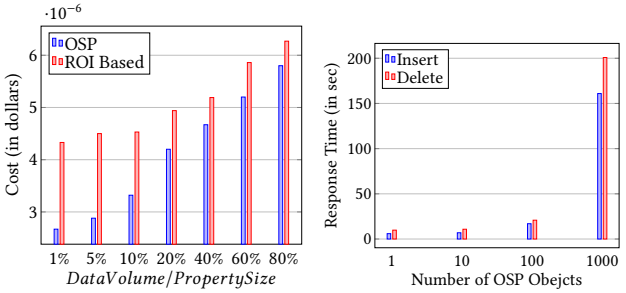


(a) Performances of Methods Varying α (b) Costs of Methods Varying α (c) Hit Ratio Varying α

Figure 8: Test for α

property requires the generation of a corresponding SPO or OSP S3 object, this significantly impacts the performance of maintenance, as illustrated in Figure 9(b), and consequently increase the cost of maintenance.

Therefore, such secondary S3 object optimizations can only be applied to low-selectivity data that are not covered by *ROAX* or to part of high-selectivity data selected by other methods or systems. Consequently, this type of optimization is not discussed as a core component in this paper.



(a) Cost of OSP and ROAX-based query varying Size (b) Performance of Maintenance varying Number of OSP

Figure 9: OSP Performance and Impact

4.5 Maintenance

Maintenance of the storage layer involves handling dynamic updates, including the insertion and deletion of triples. The maintenance of data encompasses updates to both the data files and the index.

In principle, adding triples in HuGraph is straightforward at the storage level, system downloads the corresponding data object and index object based on their property to insert triples. Then generate the new object files upload to S3. Specifically, inserting triples requires updating the records in *ROAX* that correspond to s and o, as well as adjusting the StartOffset and EndOffset of the affected records.

Based on the insert process, the deletion process introduces an optimization to improve performance. In general, the triples to be deleted in the data file are set to an invalid integer ID as a placeholder, thereby avoiding modifications to the index files. Since a placeholder is used when deleting triples, the EndOffset and StartOffset in the index can remain unmodified. Consequently, the maintenance of the index only involves insertion operations of triples.

5 COMPUTATION LAYER

In this section, we provide a detailed discussion of the three key phases of query processing: query decomposition, query plan generation, and intermediate result join.

5.1 Query Decomposition

Since the RDF graph is stored as S3 objects, the query must first be transformed into operators on these S3 objects. Since all triples sharing the same property are grouped into a single S3 object, the transformation involves decomposing the query into subqueries, each consisting of triple patterns with the same property. This ensures that the relevant triples for each subquery can be obtained in a single access operation.

Formally, given a query Q, its decomposition is a set of subqueries $\mathcal{D} = \{q_1, q_2, ..., q_t\}$ where each subquery q_i contains all triple patterns that share a specific property.

Example 5.1. *As shown in Figure 10, the query Q is divided into four subqueries $\mathcal{D} = \{q_1, q_2, q_3, q_4\}$, corresponding to four properties: deathDate, mainInterest, birthDate, and influencedBy. Among them, the query targets of (?x influencedBy ?y) and (?y influencedBy ?x) refer to the same S3 object. Thus, they are merged into a single subquery during the query decomposition. After the access method selection described in Sec.5.2, each subquery requests data from relevant S3 object and the results are joined to produce the final result.*

5.2 Access Method Determination

After the query is decomposed into subqueries, each subquery can be answered with a single access method. As discussed in Section 4.2, there are six access methods available for access. In this subsection, we explore how to determine the optimal access method for each subquery.

5.2.1 Problem Formulation and Cost Model. Given a query decomposition $\mathcal{D} = \{q_1, q_2, ..., q_t\}$, the problem is to determine the appropriate access method for each q_i.

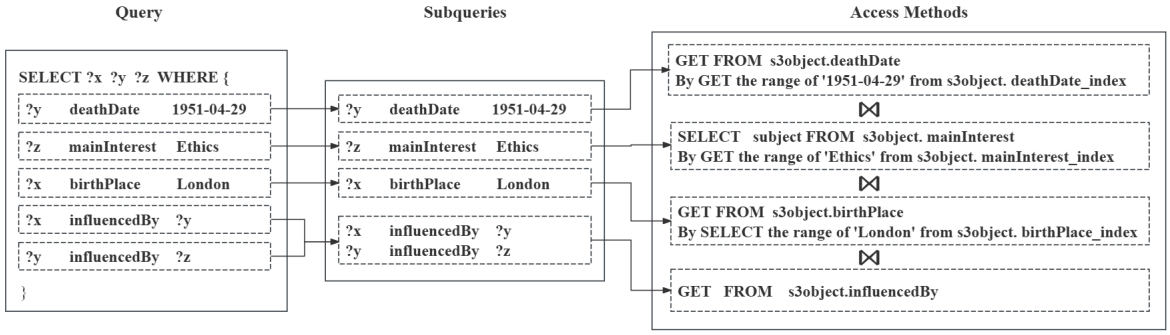


Figure 10: Example query decomposition and access method determination

For each subquery q_i, we define six boolean variables: $x^{DG}(q_i)$, $x^{DS}(q_i)$, $x^{GG}(q_i)$, $x^{GS}(q_i)$, $x^{SG}(q_i)$ and $x^{SS}(q_i)$, each corresponding to one access method, and only one of which can be set to 1, i.e., $\sum_{a\in\{DG,DS,GG,GS,SG,SS\}} x^a(q_i) = 1$.

The cost of processing a subquery q_i is composed of two components: execution time and cost. We denote the execution time of q_i under the six access methods as $TC^a(q_i)$ and its costs as $EC^a(q_i)$ ($a \in \{DG, DS, GG, GS, SG, SS\}$).

According to the pricing rules of AWS S3, both the time and costs of a subquery are primarily determined by the volume of scanned data. If a subquery involving property p is processed via the GET operator, the data scanned consists of all triples with property p. Suppose the subquery is processed via the SELECT operator or ROAX. In that case, the byte offsets of the subject or object values can be retrieved, allowing the estimation of the size of the returned data. For other cloud platforms, the pricing rules are similar to AWS, and the costs of a subquery can be estimated in a similar way.

Given a subquery q_i, its time cost $TC(q_i)$ can be expressed as follows:

$$TC(q_i) = \sum_{a\in\{DG,DS,GG,GS,SG,SS\}} x^a(q_i) \times TC^a(q_i) \tag{15}$$

and its economic cost can be formalized as follows:

$$EC(q_i) = \sum_{a\in\{DG,DS,GG,GS,SG,SS\}} x^a(q_i) \times EC^a(q_i) \tag{16}$$

The costs of a query Q can be computed as the sum of the costs of its subqueries $\{q_1, q_2, ..., q_t\}$:

$$TC(Q) = \sum_{1\le i\le t} TC(q_i);\ EC(Q) = \sum_{1\le i\le t} EC(q_i) \tag{17}$$

In real applications, users choose to minimize $EC(Q)$ as long as $TC(Q)$ can be bounded (i.e., a time constraint). We follow this approach and our optimization objective is to identify the most economical set of access methods for Q, while ensuring that TC is within the given time constraint $\overline{TC(Q)}$.

Then, our optimization problem can be formulated as follows:

Definition 5.1. ***(Formulation of Access Method Determination)*** *Given a query Q and its decomposition $\mathcal{D} = \{q_1, q_2, ..., q_t\}$, the formulation of access method determination is given as*

$$\min EC(Q)$$

s.t.

$$x^a(q_i) \in \{0, 1\}, \forall a \in \{DG, DS, GG, GS, SG, SS\}$$

$$\sum_{a\in\{DG,DS,GG,GS,SG,SS\}} x^a(q_i) = 1$$

$$TC(Q) \le \overline{TC(Q)}$$

The solution to the access method determination problem involves finding the optimal assignment of the six binary variables for each subquery q_i. The problem of access method determination is formulated as a binary integer programming problem, which is known to be NP-complete [11].

Example 5.2. *Figure 10 illustrates an example solution for determining access methods for the query shown in Figure 2. The query consists of five triple patterns, two of which share the same property, influencedBy, resulting in four distinct properties. Consequently, the query is decomposed into four subqueries, each corresponding to one unique property. In this example, we assume that each subquery is assigned a single access method. Specifically, the first, second and third subqueries are evaluated using* GG, SG, *and* GS *separately, and the last using* DG *access method.*

5.2.2 Our Solution. Although the exact solution to the optimization problem is NP-complete, SPARQL query graphs in real-world applications typically contain 10 or fewer edges [10]. Consequently, we employ a branch-and-bound algorithm to compute the globally optimal solution.

We begin by constructing a search tree, where each node represents a partial assignment of the binary variables of Definition 5.1. The tree is explored using a depth-first strategy. A *complete solution* is a full assignment of all binary variables, whereas a *partial solution* assigns only a subset of them. The search starts with all variables unassigned and proceeds by assigning variables one at a time, thereby generating a sequence of partial solutions.

For each partial solution, we compute upper ($\overline{EC(Q)}$) and lower ($\underline{EC(Q)}$) bounds on the objective value. At each step, we select the deepest unpruned leaf node—that is, the node farthest from the root—with the smallest upper bound for further branching. This

process continues until all binary variables have been considered, guaranteeing that the final solution is globally optimal.

Given a partial solution, let $\mathcal{D}'$ denote the set of subqueries whose corresponding access methods are still undetermined. The upper bound on the economic cost ($\overline{EC(Q)}$) and the lower bound on the time cost ($\underline{TC(Q)}$) of the partial solution are obtained by assuming that all subqueries in $\mathcal{D}'$ use the SELECT operator. This is because the SELECT operator incurs a substantially higher economic cost than the GET operator and the index-accelerated GET operations, while its time cost is lower. Formally,

$$\overline{EC(Q)} = \sum_{q_i \in \mathcal{D}'} EC^{SS}(q_i);\ \underline{TC(Q)} = \sum_{q_i \in \mathcal{D}'} TC^{SS}(q_i) \tag{18}$$

Conversely, the lower bound ($\underline{EC(Q)}$) for the partial solution corresponds to the scenario where all subqueries in $\mathcal{D}'$ use the GET operator. This is because the cost of the GET operator is significantly lower than the costs of the SELECT and Range SELECT operator via the index. The lower bound is computed as:

$$\underline{EC(Q)} = \sum_{q_i \in \mathcal{D}'} EC^{G}(q_i) \tag{19}$$

Algorithm 1 outlines the branch-and-bound access method determination algorithm. It initializes the global upper bound to the cost of using SELECT for all subqueries—the most expensive option (line 1). Then, a priority queue is initialized to store partial solutions, ranked by their upper bounds (line 2). The algorithm proceeds by dequeuing the partial solution with the smallest upper bound (line 4). If this solution is complete, it is added to the set of complete solutions (lines 5–7). Otherwise, the partial solution is expanded into multiple partial solutions by selecting a subquery q for which the access method has not yet been determined and setting $x^{DG}(q), x^{DS}(q), x^{GG}(q), x^{GS}(q), x^{SG}(q)$ and $x^{SS}(q)$ to 1 respectively (lines 8–16). For each partial solution, the lower and upper bounds on the time and economic costs are computed, and only partial solutions whose lower-bound time cost does not exceed the time constraint C are enqueued. If a partial solution's lower bound exceeds the current upper bound, it is pruned (lines 17–19). This process continues until all partial solutions are processed, and the complete solution with the smallest cost is returned.

In terms of time complexity, the worst-case scenario arises when pruning is ineffective, forcing the algorithm to explore the entire search space. In such cases, the time complexity is $O(6^{|\mathcal{D}|})$, as there are six variables corresponding to the six access methods, and each subquery requires only one access method to retrieve its results.

Algorithm 1: Access Method Determination Algorithm

Input: A query Q and its decomposition $\mathcal{D} = \{q_1, q_2, ..., q_t\}$
Output: An assignment of binary variables $x^G(q_i)$, $x^S(q_i)$ and $x^I(q_i)$ for each subquery q_i

1 Initialize $minUpper$ as the cost, $\overline{EC_{max}(Q)}$, of using SELECT for all subqueries.
2 Enqueue an empty partial solution into the priority queue PQ, where partial solutions are ordered by upper bounds of economic costs.
3 **while** $PQ \neq \emptyset$ **do**
4 Dequeue s_{min} with the smallest cost upper bound from PQ, and let $\mathcal{D}'_{min}$ be the set of subqueries for which the corresponding access methods are undetermined;
5 **if** s_{min} *is a complete solution* **then**
6 Insert s_{min} into CS;
7 **continue**;
8 Select one subquery q from $\mathcal{D}'_{min}$;
9 Generate six partial solutions by setting $x^{DG}(q)$, $x^{DS}(q)$, $x^{GG}(q)$, $x^{GS}(q)$, $x^{SG}(q)$ and $x^{SS}(q)$ to 1;
10 **for** *each partial solution* s *in the above* **do**
11 Calculate $s.\overline{EC(Q)}$ and $s.\underline{TC(Q)}$ using Equation 18;
12 Calculate $s.\underline{EC(Q)}$ using Equation 19;
13 **if** $s.\underline{TC(Q)} \leq C$ **then**
14 Enqueue s into PQ;
15 **if** $s.\overline{EC(Q)} < minUpper$ **then**
16 $minUpper \leftarrow s.\overline{EC(Q)}$;
17 **for** *each partial solution* s' *in* PQ **do**
18 **if** $s'.\underline{EC(Q)} > minUpper$ **then**
19 Remove s' from PQ;
20 Find the solution s_f with the smallest cost from CS;
21 **return** s_f;

5.3 Query Optimization and Execution

After the access methods for each subquery q_i have been determined, HuGraph issues corresponding requests to S3 to retrieve the results $[\![q_i]\!]$ for q_i. Finally, the results of all subqueries are joined together to form the final results.

In this paper, we directly leverage existing techniques for query optimization and execution. Specifically, we employ a System-R-style dynamic programming algorithm [6] to identify the optimal join order. This approach iteratively constructs optimal plans for subsets of subqueries: it first determines the best plan for any subset of size $k-1$ and then evaluates joining these partial results with the remaining k-th subquery. The cost of each candidate plan is estimated based on the cardinality of the intermediate results. Finally, the query is executed according to the plan with the minimal estimated cost.

Additionally, the storage layer of HuGraph is based on object storage cloud services, and query performance is primarily determined by the latency of object access and the volume of data transfer. When retrieving data using range parameters through intermediate result sets for ROAX, both matching and non-matching data are transferred simultaneously. Therefore, the size of the intermediate result set does not significantly improve query performance, even if a triple pattern is highly selective (producing few intermediate results).

Further, some classical query optimizations can also integrate with HuGraph, such as prioritizing the most selective triple patterns and iteratively using its intermediate results to retrieve data for other triples via *RAOX*. However, such strategy require multiple range scans or partial downloads on the same data file. While *RAOX* is primarily designed to reduce the amount of scanned and downloaded data, it cannot entirely avoid fetching non-matching data. Because object access latency and the number of accesses dominate

query performance, such strategies yield marginal performance gains yet increase cost through repeated accesses.

5.4 Handling Complex Queries

Thus far, our discussion has focused on Basic Graph Patterns (BGPs). In this section, we extend the framework to support complex SPARQL queries featuring operators such as **`UNION`**, **`OPTIONAL`**, and **`FILTER`**, as well as features introduced in SPARQL 1.1.

Conceptually, BGPs serve as the fundamental building blocks of SPARQL. Most complex operators can be evaluated via post-processing of BGP results. For instance, **`UNION`** corresponds to the union of two BGP results; **`OPTIONAL`** is implemented as a left outer join; and **`FILTER`** applies a predicate over the results of a BGP.

The current implementation of HuGraph supports the majority of operators defined in SPARQL 1.0 and a subset of those introduced in SPARQL 1.1, enabling robust handling of diverse query workloads.

6 IMPLEMENTATION

In this section, we present the implementation details of HuGraph, including the storage file format, data transfer, and the computation-layer implementation.

Storage Format. Since cloud object storage services such as Azure Blob Storage and Alibaba Cloud Object Storage Service (OSS) only support `SELECT`-like services for CSV files, we adopt CSV-formatted object files for data storage to ensure compatibility across different cloud environments.

Data Transfer. During data retrieval and upload, the system leverages the streaming and parallelization mechanisms provided by object storage services to improve performance within the limits of the available bandwidth and computational resources.

Data Processing. The compute layer is implemented in C++ and built on the Apache Arrow framework. After data is retrieved from object storage, the system uses the Arrow CSV Reader to read the data in parallel streams and convert it into the Arrow Table format. Data operations are executed by invoking Apache Arrow's core functions, enabling in-memory columnar data processing.

7 EVALUATION

We empirically evaluate the performance of HuGraph using multiple datasets and benchmarks. We include comparative evaluations with baselines and competitive systems.

7.1 Setting

7.1.1 Datasets. We use two real datasets and multiple synthetic datasets with different sizes in our experiments. Table 2 summarizes the statistics of these datasets.

DBpedia [17] is a real-world RDF dataset that extracts structured information from Wikipedia. In our experiments, we use the 2014 version, which contains approximately 1.1 billion triples. Additionally, DBpedia provides an open query log [10], which we utilize to derive realistic query workloads for our experiments.

Table 2: Datasets

Dataset	#Triples	#Properties	#Entities	Nature
DBpedia	1,111,481,066	124,034	139,493,254	Real
Wikidata	969,496,651	5,419	139,999,052	Real
WatDiv 100M	109,806,750	86	5,212,745	Synthetic
WatDiv 500M	549,364,969	86	26,060,373	Synthetic
WatDiv 1B	1,099,208,068	86	52,120,745	Synthetic

Wikidata [31] is a freely available knowledge base derived from Wikipedia and published as an RDF dataset. We use a version containing approximately 969 million triples. Wikidata also offers an open query log [9], which we leverage in our experiments.

WatDiv [4] is a widely used benchmark for evaluating RDF stores. We use the WatDiv generator to create datasets ranging from 100 million to 1 billion triples. The benchmark also includes a query generator that creates workloads by instantiating predefined templates with actual RDF terms extracted from the dataset.

For all datasets, we randomly sample 1,000 BGP (Basic Graph Pattern) queries from their respective query logs to form the basis of our experimental workload.

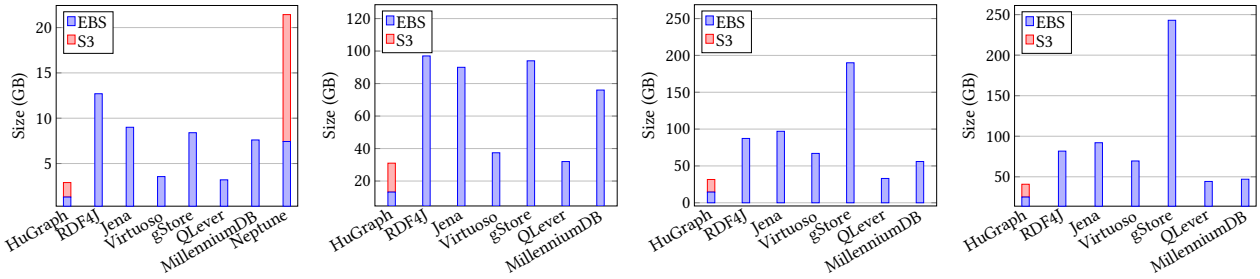
7.1.2 Experimental Environment. To ensure a uniform experimental setup, all experiments are conducted within the AWS cloud environment, using Amazon Linux 2023.3.20240304 as the server operating system. The cost calculations are based on the unified pricing of AWS, as shown in Table 1. Due to the fact that the specification used in each experiment varies depending on the size of the dataset, the specific environment for each RDF system is fixed as shown in Table 3. The implementation is in C++, and the source code is publicly accessible via GitHub[3].

As mentioned in Section 4.2, the index requires a threshold parameter α that controls the strictness of the condition to achieve higher efficiency. By default, we set $\alpha = 50\%$ as the default configuration, with the specific reasons elaborated in Section 4.3.

7.1.3 Competitors. We compare HuGraph in terms of time and economic costs against Amazon cloud-based graph database service Neptune [8] and six well-known centralized RDF stores deployed on the Amazon EC2 instances: gStore [36], Jena [16], RDF4J [2], Virtuoso [13], QLever [7] and MillenniumDB [32]. These provide cloud-based interfaces for supporting SPARQL queries. To support SPARQL queries, an RDF graph is pre-processed for storage as follows:

(1) Jena [16], RDF4J [2], gStore [36], Virtuoso [13], QLever [7] and MillenniumDB [32] initialize their stores on AWS EBS. Each store retrieves the dataset files from AWS EBS and loads the data into its own store, also hosted on AWS EBS, while employing indexing mechanisms specifically designed for its architecture.
(2) The operation of Neptune [8] is dependent on the S3 storage service. Thus, the dataset files are first uploaded to an S3 bucket and then Neptune's `load` command is executed. This command initiates the process of loading the data onto the disk of Neptune's operational server.

[3] https://anonymous.4open.science/r/HuGraph-0FD6/



(a) WatDiv 100M (b) WatDiv 1B (c) DBpedia (d) Wikidata

Figure 11: Storage size

Table 3: Economical settings for different RDF stores

Systems	WatDiv-100	WatDiv-500	WatDiv-1B	DBpedia	Wikidata
HuGraph	2C/4GB	2C/8GB	2C/8GB	2C/8GB	2C/8GB
RDF4J	2C/4GB	–	8C/32GB	2C/8GB	4C/16GB
Jena	2C/4GB	–	2C/8GB	2C/8GB	2C/8GB
Virtuoso	2C/4GB	–	2C/8GB	4C/16GB	4C/16GB
gStore	8C/32GB	–	32C/128GB	64C/256GB	32C/128GB
QLever	2C/8GB	–	2C/8GB	4C/16GB	4C/16GB
MillenniumDB	2C/8GB	–	4C/16GB	4C/16GB	4C/16GB

(3) For HuGraph, the RDF graphs, the indices and two mapping dictionaries are loaded into the S3 storage service as described earlier.

7.2 Offline Performance

7.2.1 System Configurations. We configure the environment for these systems to match their optimal settings for each dataset and its associated queries. This ensures that each system can complete the experiments using the most resource-efficient setup. To achieve this, we evaluate each system under a variety of computational resource configurations while executing the query workloads. Our goal is to identify the minimum resource requirements that allow the RDF stores to successfully execute all queries.

The results of these experiment for all five datasets can be found in Table 3. HuGraph is capable of running all queries with minimal computational resources across all five datasets. Since gStore requires loading all data into memory for query acceleration, it needs the most computational resources.

7.2.2 Comparison of Storage. We compare the storage cost of different systems – see Figure 11 for the storage requirement and Figure 12 for storage cost per month for all datasets, based on their storage use and the AWS service pricing in Table 1. HuGraph reduces storage cost across three large datasets: WatDiv-1B, DBpedia, and Wikidata, achieving at least a 50% reduction. This is mainly because the EBS storage service is 3.47× more expensive than the S3 storage service, and HuGraph replaces all IRIs and literals with unique integer IDs.

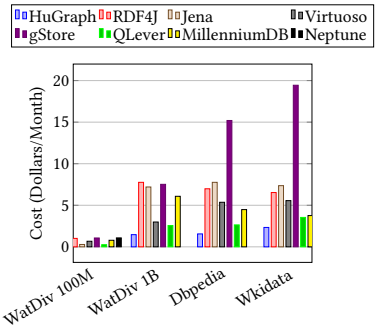


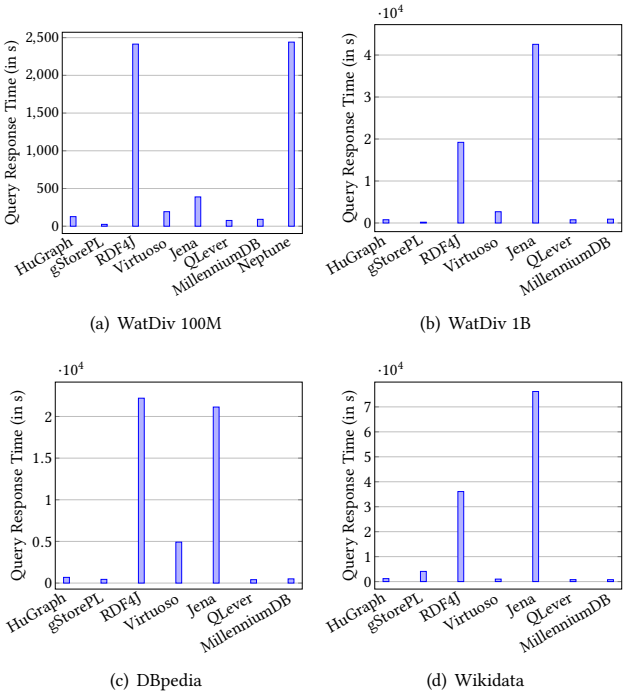
Figure 12: Storage economic cost

7.3 Efficiency Test

This experiment evaluates the performance and cost of HuGraph in comparison to competitive systems. For each system, we sequentially execute the queries while recording both execution time and associated costs. As noted above, gStore requires loading indices into memory prior to query execution, with loading time being proportional to graph size. To ensure a comprehensive comparison, we evaluate HuGraph against gStorePL (pre-loaded), excluding loading time.

Figure 13 reports the query execution times across all evaluated systems. Among them, HuGraph, gStorePL, QLever, and MillenniumDB constitute the top-performing tier. However, when interpreted alongside Figure 14, which illustrates the corresponding resource costs, HuGraph exhibits a compelling advantage: it delivers robust performance and superior economic efficiency in large-scale query processing, avoiding the prohibitive resource overheads incurred by its fastest competitors.

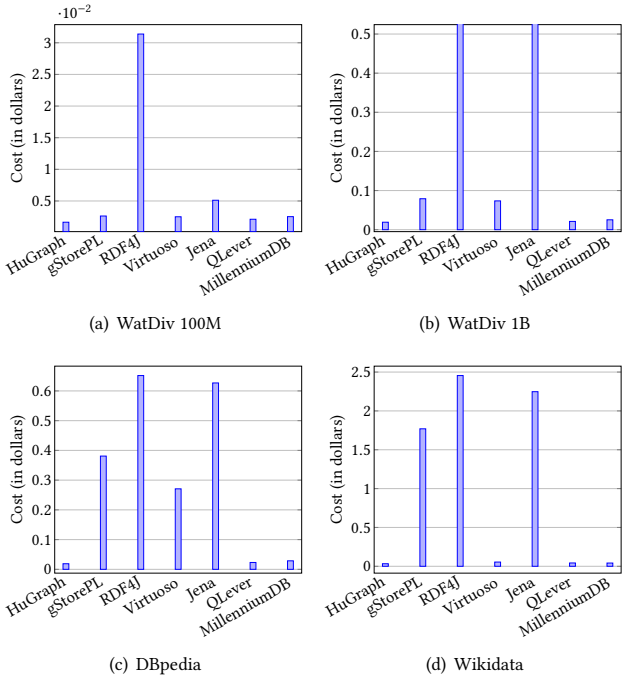
Specifically, HuGraph demonstrates consistently strong performance across all datasets, although it does not always attain the absolute minimum runtime. On most datasets, gStorePL and QLever surpass HuGraph in raw query speed. This performance edge, however, stems from their aggressive in-memory data pre-loading, which incurs substantially higher memory footprints and thus mandates more powerful—and significantly more expensive—hardware configurations. For example, on DBpedia, HuGraph completes the full query workload in 683 seconds using a lightweight EC2 instance with only 2 vCPUs and 8 GB of memory. In comparison, gStorePL



(a) WatDiv 100M (b) WatDiv 1B

(c) DBpedia (d) Wikidata

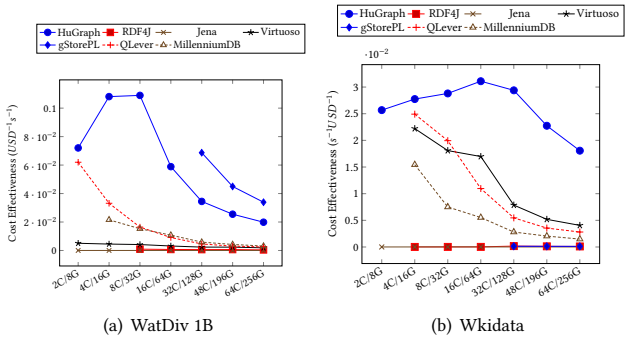
Figure 13: Query performance



(a) WatDiv 100M (b) WatDiv 1B

(c) DBpedia (d) Wikidata

Figure 14: Query cost



(a) WatDiv 1B (b) Wkidata

Figure 15: Cost Effectiveness

and QLever finish the same workload in 443 seconds and 407 seconds, respectively, but require far more substantial resources: 64 vCPUs with 256 GB of memory for gStorePL, and 4 vCPUs with 16 GB of memory for QLever. Moreover, relative to traditional systems such as Virtuoso, Jena, and RDF4J, HuGraph consistently offers both better performance and lower economic costs.

We also conduct the same experiment on the smaller WatDiv 100M dataset, adding the Neptune system for comparison. The results indicate that the Neptune database shows no significant advantage in terms of either performance or cost. Neptune mandates deployment within a Virtual Private Cloud (VPC), and its total cost comprises four distinct components: compute capacity, storage volume, storage I/O operations, and an EC2 instance required to interface with the Neptune service. Collectively, these factors lead to a substantially higher overall cost.

7.4 Cost Effectiveness Test

In this experiment, we evaluate query performance on the WatDiv 1B and Wikidata datasets under varying configurations to assess the cost effectiveness of each system. We define *cost-effectiveness* as the ratio of performance to economic cost, formulated as $\frac{1/TC}{EC} = \frac{1}{TC \times EC}$, where performance is measured as the reciprocal of response time. Thus, a system with shorter response time and lower economic cost achieves higher cost-effectiveness.

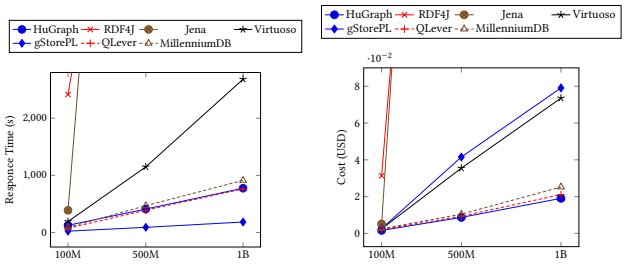
As shown in Figure 15, the cost effectiveness of all systems declines as computational resources become increasingly over-provisioned. For HuGraph, as mentioned in Section 6, the streaming data ingestion mechanism limits overall processing efficiency under low-resource configurations, while parallel data transfer substantially improves query processing efficiency as resources increase. However, further resource increases lead to over-provisioning for a fixed data scale, causing cost effectiveness to decline. Despite this trend, HuGraph achieves the highest cost effectiveness by maintaining good performance under resource-constrained configurations and minimizing query costs through low-cost storage and access method determination.

7.5 Scalability Test

In this experiment, we compare HuGraph with other systems on the WatDiv 100M, 500M, and 1B datasets. We then analyze the relationships between performance and data volume, as well as

between cost and data volume, by measuring the behavior of the systems with varying dataset sizes.

Experimental results in Fig. ?? show that both the query processing time and the actual cost increase linearly as the amount of data increases. These results confirm that HuGraph scales to large graphs, and that the rate of increase as the dataset size grows is slow.



(a) Performance with Data Volume (b) Cost with Data Volume

Figure 16: Scalability Test

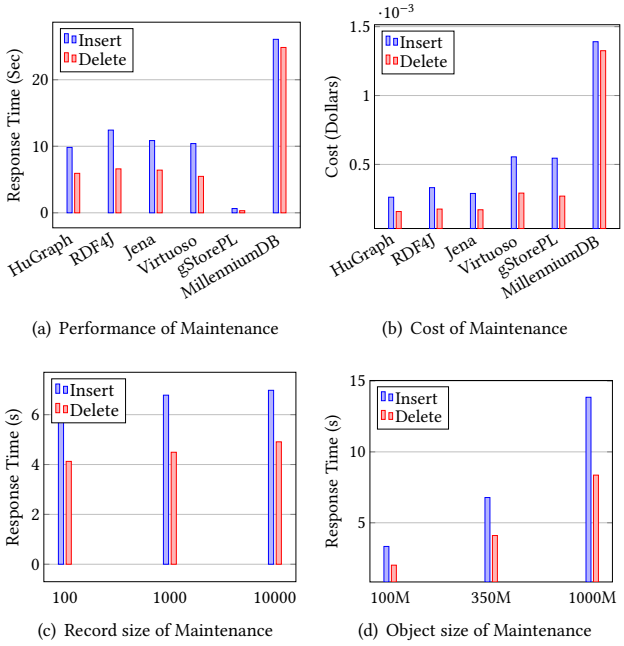
7.6 Evaluation of Maintenance

In this experiment, we utilize the DBpedia dataset to evaluate the efficiency of maintenance operations in HuGraph. First, we assess deletion and insertion performance through the following procedure: First, we randomly sample 1000 of the triples and delete them to measure deletion throughput. Subsequently, we re-insert this subset of triples to evaluate insertion performance and cost. Considering that the maximum acceptable length of a single statement varies across databases, we set the operation size to 100 triples per insertion or deletion, and repeat this operation 10 times to complete the addition or removal of 1000 triples in total. Moreover, since Qlever is a query engine that does not support maintenance operations, it is excluded from this experiment.

Then, we evaluated the impact of record count by performing insertion and deletion operations on 100, 1000, and 10000 records in a 350 MB object file. To evaluate the impact of object size on maintenance overhead, we performed the same operations on 1000 records in object files of 100 MB, 350 MB, and 1000 MB.

Figure 17(a) presents the performance first experimental results. gStore-L still exhibits a significant advantage over all other database systems, while HuGraph maintains competitive performance. In terms of cost, however, as shown in Figure 17(b), gStore-L incurs high expenses due to its stringent EC configuration requirements. In contrast, owing to its lightweight resource demands and competitive performance, HuGraph achieves the best cost-effectiveness among all databases tested.

Figure 17(c) and Figure 17(d) illustrate the effects of record count and object file size on maintenance. Because of the object storage service's file-level operation semantics, the number of records has little impact on performance. In contrast, maintenance overhead increases noticeably with object file size, exhibiting a clear positive correlation.



(a) Performance of Maintenance (b) Cost of Maintenance

(c) Record size of Maintenance (d) Object size of Maintenance

Figure 17: Evaluation of Maintenance

7.7 Evaluation of Access Method Determination

To validate the effectiveness of our access method determination algorithm, we perform an ablation study where we execute the queries using HuGraph using both the recommended acess method (by the algorithm) and otheraccess access methods.

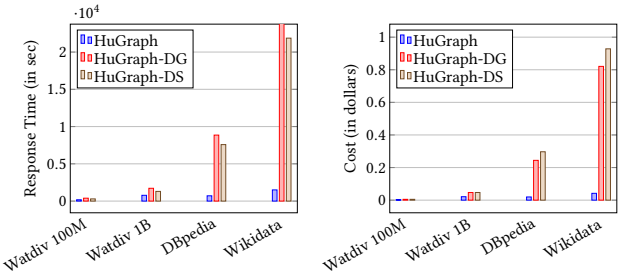
In most real queries, only a few triple patterns contain constants in the subject or object positions. Most triple patterns feature variables in both of these positions. For such patterns, the access methods base on ROAX(p) cannot determine a concrete *Value* parameter and thus cannot be applied and only the DG or DS access methods are viable.

Consequently, we design our evaluation to compare HuGraph that uses the algorithm-determined access method against HuGraph-DG, which exclusively uses the DG method, and HuGraph-DS, which relies solely on DS. The results of the experiment are given in Figure 18.On the four tested datasets, HuGraph with the algorithm-selected access method outperforms HuGraph-DS and HuGraph-DG, achieving performance speedups of up to 14.57× and 19.61×, and cost reductions of up to 22.18× and 18.61×, respectively.

7.8 Cross-Cloud Evaluation

To evaluate the generality of HuGraph across different cloud platforms, in this experiment we replace *AWS S3* and *EC2* with *Alibaba Cloud OSS* and *ECS*, repeating the storage and efficiency tests described in Section 7.2.2 and Section 7.3 using the WatDiv 100M dataset. The specific environment for each RDF stores on Alibaba Cloud matches exactly the configuration specified in Table 3.

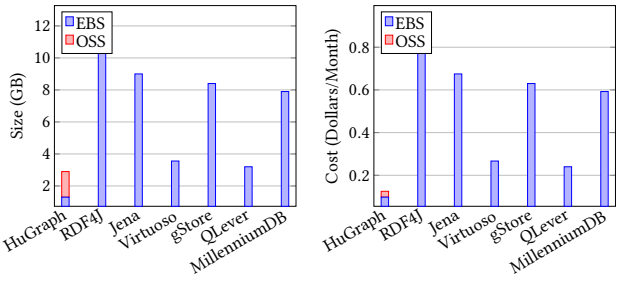
The conversion between RMB (for Alibaba Cloud) and U.S. dollars is calculated using an exchange rate of 1:7. Figure 19 presents the storage consumption and cost results for all RDF stores. The



(a) Performance comparison (b) Cost comparison

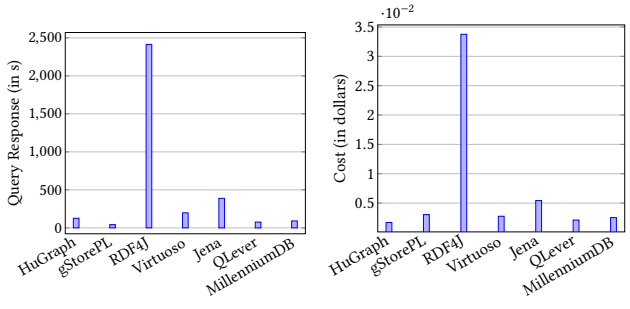
Figure 18: Evaluation of access methods determination

storage usage remains unchanged across systems, and although the cost varies due to different pricing models in the cloud environments, HuGraph is still the most cost-efficient. Figure 20 reports the query performance and cost results. Despite numerical differences caused by the performance characteristics of the respective cloud environments, HuGraph consistently maintains the lowest cost with 6% and 15% advantages compared with Virtuoso and gStorePL.



(a) Stoarge size (b) Storage economic cost

Figure 19: Stoarge test in Alibaba cloud



(a) Query performance (b) Query cost

Figure 20: Efficiency test in Alibaba cloud

8 RELATED WORK

Although many RDF graph systems exist, very few are designed to be cloud-native. In this paper, we focus primarily on cloud-native RDBMSs.

Although there are a number of cloud-native DBMSs (e.g., Amazon Aurora [29, 30], Presto[27], Snowflake [12], PushdownDB [34], FlexPushdownDB [33], Redshift Spectrum[5] and Iceberg[1]), these systems are specifically optimized for relational data storage and querying under a disaggregated storage-and-compute architecture tailored to cloud environments. However, none provide native support—or substantial discussion—of RDF graph querying via SPARQL.

Only a few cloud-native DBMSs claim that they can support graph queries. AWS Neptune[8] is a fully managed graph database service provided by Amazon. It adopts a disaggregated storage-and-compute architecture and supports RDF storage and querying, with excellent scalability. However, because Neptune only supports network queries within the same VPC, using the Neptune service requires simultaneous use of AWS EC2, Neptune, and S3 services, resulting in relatively high usage costs. X-Stor [18] is a cloud-native, multi-model NoSQL database system designed to support the storage and management of multiple data models, including the graph data model. However, its cloud service for graph data has not been tested either in the paper describing it nor made publicly available.

The core ideas underlying our approach—organizing RDF data by property and decomposing queries accordingly—have precedents in prior systems that employ *vertical partitioning*: triples sharing the same property are grouped into the same partition. This strategy is widely adopted in big data–based RDF engines such as HadoopRDF [15], CliqueSquare [14], S2RDF [23], WORQ [19], and Sparklify [25]. In these systems, each property corresponds to a distinct HDFS file or Spark SQL table. In contrast, our approach stores triples of the same property within a single cloud object (e.g., in AWS S3) to minimize storage costs under a disaggregated architecture. We then design storage-aware indices to accelerate selective access—striking a balance between cost and performance.

In addition, our query decomposition strategy also parallels techniques used in federated RDF query engines like DARQ [22] and FedX [24], which also decompose SPARQL queries by triple pattern and select relevant endpoints for the subqueries. In those systems, triple patterns mapping to the same single endpoint can be merged into a single subquery. In our setting, however, each triple pattern typically maps to a distinct S3 object (since objects group triples by property), making such merging infeasible. Consequently, a key challenge in our approach lies in selecting the optimal access method—e.g., full object scan vs. indexed retrieval—for each subquery under cost-performance trade-offs.

9 CONCLUSIONS

We introduce HuGraph, a cloud-native RDF store that leverages a disaggregated compute-storage architecture and cloud-optimized access methods. Designed to minimize RDF store economic costs while maintaining acceptable performance, HuGraph stores RDF data in remote object storage service based on the properties, where each object is accompanied by some index files. Furthermore, we design an index, called *Range-Aware Offset Index*, that combines

the GET and SELECT operator, optimizing both economic cost and processing efficiency. During query execution, HuGraph decomposes SPARQL queries into subqueries and selects the most suitable access method for each, balancing both performance and economic cost. These selected methods are then integrated into a comprehensive query execution plan. Since access method selection is an NP-complete problem, we propose a branch-and-bound algorithm to efficiently determine optimal access strategies. Experiments show that, compared to RDF stores with either compute-storage integrated or disaggregated architectures, HuGraph maintains competitive performance while achieving better economic cost. The system has been fully implemented and is publicly available.

REFERENCES

[1] 2024. "Apache Iceberg,". https://iceberg.apache.org/

[2] 2024. Eclipse RDF4J. https://rdf4j.org/

[3] Daniel J. Abadi, Adam Marcus, Samuel R. Madden, and Kate Hollenbach. 2009. SW-Store: a vertically partitioned DBMS for Semantic Web data management. *The VLDB Journal* 18, 2 (April 2009), 385–406.

[4] Güneş Aluç, Olaf Hartig, M. Tamer Özsu, and Khuzaima Daudjee. 2014. Diversified Stress Testing of RDF Data Management Systems. In *The Semantic Web – ISWC*. Springer International Publishing, Cham, 197–212.

[5] Nikos Armenatzoglou, Sanuj Basu, Naga Bhanoori, Mengchu Cai, Naresh Chainani, Kiran Chinta, Venkatraman Govindaraju, Todd J. Green, Monish Gupta, Sebastian Hillig, Eric Hotinger, Yan Leshinksy, Jintian Liang, Michael McCreedy, Fabian Nagel, Ippokratis Pandis, Panos Parchas, Rahul Pathak, Orestis Polychroniou, Foyzur Rahman, Gaurav Saxena, Gokul Soundararajan, Sriram Subramanian, and Doug Terry. 2022. Amazon Redshift Re-invented. In *SIGMOD*. Association for Computing Machinery, New York, NY, USA, 2205–2217.

[6] M. M. Astrahan, M. W. Blasgen, D. D. Chamberlin, K. P. Eswaran, J. N. Gray, P. P. Griffiths, W. F. King, R. A. Lorie, P. R. McJones, J. W. Mehl, G. R. Putzolu, I. L. Traiger, B. W. Wade, and V. Watson. 1976. System R: relational approach to database management. *ACM Trans. Database Syst.* 1, 2 (June 1976), 97–137.

[7] Hannah Bast and Björn Buchhold. 2017. QLever: A Query Engine for Efficient SPARQL+Text Search. In *CIKM*. Association for Computing Machinery, New York, NY, USA, 647–656.

[8] Bradley R. Bebee, Daniel Choi, Ankit Gupta, Andi Gutmans, Ankesh Khandelwal, Yigit Kiran, Sainath Mallidi, Bruce McGaughy, Mike Personick, Karthik Rajan, Simone Rondelli, Alexander Ryazanov, Michael Schmidt, Kunal Sengupta, Bryan B. Thompson, Divij Vaidya, and Shawn Wang. 2018. Amazon Neptune: Graph Data Management in the Cloud. In *ISWC (CEUR Workshop Proceedings)*, Vol. 2180. CEUR-WS.org.

[9] Angela Bonifati, Wim Martens, and Thomas Timm. 2019. Navigating the Maze of Wikidata Query Logs. In *The World Wide Web Conference*. Association for Computing Machinery, New York, NY, USA, 127–138.

[10] Angela Bonifati, Wim Martens, and Thomas Timm. 2020. An analytical study of large SPARQL query logs. *The VLDB Journal* 29, 4 (2020).

[11] Harlan P. Crowder, Ellis L. Johnson, and Manfred W. Padberg. 1983. Solving Large-Scale Zero-One Linear Programming Problems. *Operations Research* 31, 5 (1983), 803–834.

[12] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *SIGMOD*. Association for Computing Machinery, New York, NY, USA, 215–226.

[13] Orri Erling. 2012. Virtuoso, A Hybrid RDBMS/Graph Column Store. *IEEE Data Eng. Bull.* 35, 1 (2012), 3–8.

[14] François Goasdoué, Zoi Kaoudi, Ioana Manolescu, Jorge-Arnulfo Quiané-Ruiz, and Stamatis Zampetakis. 2015. CliqueSquare: Flat Plans for Massively Parallel RDF Queries. In *Proc. ICDE*. 771–782.

[15] Mohammad Farhan Husain, James P. McGlothlin, Mohammad M. Masud, Latifur R. Khan, and Bhavani M. Thuraisingham. 2011. Heuristics-Based Query Processing for Large RDF Graphs Using Cloud Computing. *TKDE* 23, 9 (2011), 1312–1327.

[16] Vaibhav Khadilkar, Murat Kantarcioglu, Bhavani M. Thuraisingham, and Paolo Castagna. 2012. Jena-HBase: A Distributed, Scalable and Effcient RDF Triple Store. In *International Workshop on the Semantic Web*.

[17] Jens Lehmann, Robert Isele, Max Jakob, Anja Jentzsch, Dimitris Kontokostas, Pablo N. Mendes, Sebastian Hellmann, Mohamed Morsey, Patrick van Kleef, Sören Auer, and Christian Bizer. 2015. DBpedia - A Large-scale, Multilingual Knowledge Base Extracted from Wikipedia. *Semantic Web* 6, 2 (2015), 167–195.

[18] Hongyu Lei, Chunhua Li, Ke Zhou, Jianping Zhu, Kezhou Yan, Fen Xiao, Ming Xie, Jiang Wang, and Shiyu Di. 2024. X-Stor: A Cloud-Native NoSQL Database Service with Multi-Model Support. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4025–4037.

[19] Amgad Madkour, Ahmed M. Aly, and Walid G. Aref. 2018. WORQ: Workload-Driven RDF Query Processing. In *Proc. ISWC*. 583–599.

[20] Thomas Neumann and Gerhard Weikum. 2010. The RDF-3X engine for scalable management of RDF data. *VLDB J.* 19, 1 (2010), 91–113.

[21] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. 2009. Semantics and complexity of SPARQL. *ACM Trans. Database Syst.* 34, 3, Article 16 (Sept. 2009), 45 pages.

[22] Bastian Quilitz and Ulf Leser. 2008. Querying Distributed RDF Data Sources with SPARQL. In *ESWC*. 524–538.

[23] Alexander Schätzle, Martin Przyjaciel-Zablocki, Simon Skilevic, and Georg Lausen. 2016. S2RDF: RDF Querying with SPARQL on Spark. *PVLDB* 9, 10 (2016), 804–815.

[24] Andreas Schwarte, Peter Haase, Katja Hose, Ralf Schenkel, and Michael Schmidt. 2011. FedX: Optimization Techniques for Federated Query Processing on Linked Data. In *ISWC*. 601–616.

[25] Claus Stadler, Gezim Sejdiu, Damien Graux, and Jens Lehmann. 2019. Sparklify: A Scalable Software Component for Efficient Evaluation of SPARQL Queries over Distributed RDF Datasets. In *Proc. ISWC*. 293–308.

[26] Markus Stocker, Andy Seaborne, Abraham Bernstein, Christoph Kiefer, and Dave Reynolds. 2008. SPARQL basic graph pattern optimization using selectivity estimation. In *Proceedings of the 17th International Conference on World Wide Web*. Association for Computing Machinery, New York, NY, USA, 595–604.

[27] Yutian Sun, Tim Meehan, Rebecca Schlussel, Wenlei Xie, Masha Basmanova, Orri Erling, Andrii Rosa, Shixuan Fan, Rongrong Zhong, Arun Thirupathi, Nikhil Collooru, Ke Wang, Sameer Agarwal, Arjun Gupta, Dionysios Logothetis, Kostas Xirogiannopoulos, Amit Dutta, Varun Gajjala, Rohit Jain, Ajay Palakuzhy, Prithvi Pandian, Sergey Pershin, Abhisek Saikia, Pranjal Shankhdhar, Neerad Somanchi, Swapnil Tailor, Jialiang Tan, Sreeni Viswanadha, Zac Wen, Biswapesh Chattopadhyay, Bin Fan, Deepak Majeti, and Aditi Pandit. 2023. Presto: A Decade of SQL Analytics at Meta. *Proc. ACM Manag. Data* 1, 2, Article 189 (June 2023), 25 pages.

[28] Junjay Tan, Thanaa Ghanem, Matthew Perron, Xiangyao Yu, Michael Stonebraker, David DeWitt, Marco Serafini, Ashraf Aboulnaga, and Tim Kraska. 2019. Choosing a cloud DBMS: architectures and tradeoffs. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 2170–2182.

[29] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *SIGMOD*. Association for Computing Machinery, New York, NY, USA, 1041–1052.

[30] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, James Corey, Kamal Gupta, Murali Brahmadesam, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvilli, and Xiaofeng Bao. 2018. Amazon Aurora: On Avoiding Distributed Consensus for I/Os, Commits, and Membership Changes. In *SIGMOD*. Association for Computing Machinery, New York, NY, USA, 789–796.

[31] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[32] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. 2023. MillenniumDB: An Open-Source Graph Database System. *Data Intelligence* 5, 3 (08 2023), 560–610.

[33] Yifei Yang, Matt Youill, Matthew Woicik, Yizhou Liu, Xiangyao Yu, Marco Serafini, Ashraf Aboulnaga, and Michael Stonebraker. 2021. FlexPushdownDB: hybrid pushdown and caching in a cloud DBMS. *Proc. VLDB Endow.* 14, 11 (July 2021), 2101–2113.

[34] Xiangyao Yu, Matt Youill, Matthew E. Woicik, Abdurrahman Ghanem, Marco Serafini, Ashraf Aboulnaga, and Michael Stonebraker. 2020. PushdownDB: Accelerating a DBMS Using S3 Computation. In *ICDE*. IEEE, 1802–1805.

[35] Lei Zou, Jinghui Mo, Lei Chen, M. Tamer Özsu, and Dongyan Zhao. 2011. gStore: Answering SPARQL Queries via Subgraph Matching. *Proc. VLDB Endow.* 4, 8 (2011), 482–493.

[36] Lei Zou, M. Tamer Özsu, Lei Chen, Xuchuan Shen, Ruizhe Huang, and Dongyan Zhao. 2014. gStore: a graph-based SPARQL query engine. *VLDB J.* 23, 4 (2014), 565–590.